

Assignment - 10

Title ; File Handling

Objective : To learn different file Organizations

problem Statement :-

Department maintain's Student's database. The File Contain's roll no., name, division and address. Write a program to Create a sequential File to Store and maintain student data. It should allow the user to add, delete information of the Student. Display information of particular Student. If record of Student does not exist an appropriate message is displayed. If Student is found it should display the student details.

Out Come : Solve problem using algorithm design technique's.

Theory :-

Concept of File → File Can be defined as the Collection of record's in Which each record consist of one or more Field's

e.g :-	Roll Number	Name	Mark's	Address
	01	ABC	90	MG Road
	02	AME	40	S Nagar
	03	OBKE	50	S.k.Marg

• Various Operation's that Can be performed on the File are:-

1. Insertion of record into the File.
2. Deletion of record.
3. Updation of record.
4. Retrieval of record.

⇒ Fixed length record file :- The File in which all the records are of same length or of fixed length is called as Fixed length record File.

Advantage :- If the record size is fixed then there is fast access to any desired record. Because, the starting address and ending address of each record is known.

Disadvantage :- If the record is too large there are chances of losing some information. Similarly if the record is too short there may be wastage of some memory.

File Operation's :- C++ provides following classes to perform input and output of characters to and from the file's.

ofstream	This stream class is used to write on file's.
ifstream	This stream class is used to read from file.
fstream	This stream class is used for both read/write.

To perform file I/O, we need to include `<fstream.h>` in the program. It defines several classes, including `ifstream`, `ofstream` and `fstream`. These classes are derived from `ios`, so `ifstream`, `ofstream` and `fstream` have access to all operations defined by `ios`.

⇒ **Open File Operations:-** The file operations are associated with the object of the classes: `ifstream`, `ofstream` or `fstream`. Hence we need to create an object of the corresponding class.

The syntax of file open is :- `Open(filename, mode)`
The mode is optional parameter and is used with the flags as given below:-

`ios::in` ⇒ Open for input operation.

`ios::out` ⇒ Open for output operation.

⇒ **Close File Operation:-** To close the file the member function `close()` is used. The `close` function takes no parameter and returns no value.

e.g :- `obj.close();`

We can detect when the end of an input file has been reached by using the `eof()` member function of `ios`. It returns true when the end of the file has been encountered and false otherwise.

File types and File Organization :-

Def :- File organization deals with representation or arrangement of data on external storage.

Various type of File Organization are :-

1. Sequential file Organization
2. Index Sequential file Organization
3. Direct Access file Organization.

Sequential file Organization :- In this file organization the records are stored one by one by advancing the file pointer on each insertion of record.

Basic Operations that can be performed on Sequential file Organisation are.

- | | | |
|-----------|------------|-----------|
| 1. Create | 2. Display | 3. Delete |
| 4. Update | 5. Append | 6. Search |

Test Case	Test Case Description	Test Data	Expected Result	Actual Result	pass/fail
1.	Check the result When user select an option of Create a file to Store Student information.	Create the file to store the Student database	File get Create to Store the Student information	File get's created to Store the Student's information	Pass
2.	Check the result When user select an option of add info to File.	New student information insert into the file.	New Student info. Should get's added to the file.	New information of Student get inserted	pass.
3.	Check result When user select an option of delete existing record from File.	Existing Student's record get deleted from a file.	Database of existing Student Should get removed from the file.	Deletion of existing Student's record is get done.	pass
4.	Check's result When user select's an option of display info of a particular Student	Information of a particular Student get's displayed	User inserted Student's database Should get displayed.	User inserted Student's database get displayed on Screen	pass

ASSIGNMENT 10

```
#include <iostream>
#include <fstream>
#include <iomanip>
#define SIZE 100
using namespace std;

class Student
{
private:
    int rollno;
    int marks;
    char name[SIZE];

public:
    void accept();
    void display();
    void modify();
    int getrno();
};

void Student::accept()
{
```



```
cout << "Enter the roll number:- ";
cin >> rollno;
cin.ignore();
cout << "Enter the name of the student:- ";
cin.getline(name, SIZE);
cout << "Enter the marks:- ";
cin >> marks;
}
```

```
void Student::display()
{
    cout << "\t" << rollno << "\t " << name << " \t" << marks << "\n";
}
```

```
void Student::modify()
{
    cin.ignore();
    cout << "Enter new name of student:- ";
    cin.getline(name, SIZE);
    cout << "Enter the new marks:- ";
    cin >> marks;
}
```

```
int Student::getrno()
{
    return rollno;
}
```

```
int create(int);  
void display(int);  
int add(int);  
void deleterec(int);  
void modifyrec(int);
```

```
int main()  
{  
    int choice, i, flag, cntr = 0;  
  
    while (1)  
    {  
        cout << "\nStudents Database" << endl;  
        cout << "1. Create" << endl;  
        cout << "2. Display" << endl;  
        cout << "3. Add" << endl;  
        cout << "4. Delete" << endl;  
        cout << "5. Modify" << endl;  
        cout << "6. Exit program" << endl;  
        cout << "\nEnter your choice:- ";  
        cin >> choice;  
  
        switch (choice)  
        {  
            case 1:  
            {  
                cntr = create(cntr);  
                break;
```



```
}
```

```
case 2:
```

```
{
```

```
    display(cntr);
```

```
    break;
```

```
}
```

```
case 3:
```

```
{
```

```
    if (cntr == 0)
```

```
        cout << "\nPlease build a master table first" << endl;
```

```
    else
```

```
        cntr = add(cntr);
```

```
    break;
```

```
}
```

```
case 4:
```

```
{
```

```
    if (cntr == 0)
```

```
        cout << "\nPlease build a master table first" << endl;
```

```
    else
```

```
        deleterec(cntr);
```

```
    break;
```

```
}
```

```
case 5:
```

```
{
```

```
    if (cntr == 0)
```

```
        cout << "\nPlease build a master table first" << endl;
```

```

        else
            modifyrec(cntr);
        break;
    }
    case 6:
        return 0;
    default:
        cout << "Error in choice,try again" << endl;
    }
}
return 0;
}

```

```

int create(int cntr)
{
    int rno;
    Student m;
    ofstream fout("recs.dat", ios::binary | ios::out);
    cntr = 0;
    char ask = 'y';
    while (ask == 'y' || ask == 'Y')
    {
        m.accept();
        fout.write((char *)&m, sizeof(m));
        cntr++;
        cout << "\nDo you want to add another record?(y/n):- ";
        cin >> ask;
    }
}

```



```

    fout.close();
    return cntr;
}

void display(int cntr)
{
    int i;
    Student m;
    if (cntr == 0)
        cout << "\nPlease build a master table first" << endl;
    else
    {
        ifstream fin("recs.dat", ios::binary | ios::in);
        cout << "\n-----" << endl;
        cout << "Sr No. " << setw(5) << "Roll No. ";
        cout << " Name ";
        cout << " Marks" << endl;
        cout << "\n-----" << endl;
        i = 1;
        while (fin.read((char *)&m, sizeof(m)))
        {
            cout << " " << i << " ";
            m.display();
            i++;
        }
        fin.close();
    }
}

```

```

int add(int cntr)
{
    Student m;
    ofstream fout("recs.dat", ios::binary | ios::app);
    cout << "Enter the details of the record you want to insert:- " << endl;
    m.accept();
    fout.write((char *)&m, sizeof(m));
    fout.close();
    cntr++;
    cout << "Record inserted successfully" << endl;
    return cntr;
}

```

```

void deleterec(int cntr)
{
    Student m;
    int flag;
    int rno;
    ifstream fin("recs.dat", ios::binary | ios::in);
    cout << "Enter the roll number of the record you want to delete:- ";
    cin >> rno;
    flag = 0;
    while (fin.read((char *)&m, sizeof(m)))
    {
        if (rno == m.getrno())
        {
            flag = 1;

```

```

        cntr--;
        break;
    }
}
fin.close();
if (!flag)
    cout << "\nRecord with roll number " << rno << " not found" << endl;
else
{
    ofstream fout("temp.dat", ios::binary | ios::out);
    ifstream fin("recs.dat", ios::binary | ios::out);
    fin.seekg(0, ios::beg);
    fin.read((char *)&m, sizeof(m));
    while (!fin.eof())
    {
        if (m.getrno() != rno)
        {
            fout.write((char *)&m, sizeof(m));
            fin.read((char *)&m, sizeof(m));
        }
    }
    fout.close();
    fin.close();
    cout << "Record with roll number " << rno << " deleted successfully" << endl;
    remove("recs.dat");
    rename("temp.dat", "recs.dat");
}
}

void modifyrec(int cntr)

```



```

{
    Student m;
    int rno;
    int flag;
    ifstream fin("recs.dat", ios::binary | ios::in);
    cout << "Enter the roll number of the record you want to edit:- ";
    cin >> rno;
    flag = 0;
    while (fin.read((char *)&m, sizeof(m)))
    {
        if (rno == m.getrno())
        {
            flag = 1;
            cntr--;
            break;
        }
    }
    fin.close();
    if (!flag)
        cout << "\nRecord with roll number " << rno << " not found" << endl;
    else
    {
        ofstream fout("temp.dat", ios::binary | ios::out);
        ifstream fin("recs.dat", ios::binary | ios::out);
        fin.read((char *)&m, sizeof(m));
        while (!fin.eof())
        {
            if (m.getrno() == rno)

```

```
{
    m.modify();
    fout.write((char *)&m, sizeof(m));
}
else
    fout.write((char *)&m, sizeof(m));
    fin.read((char *)&m, sizeof(m));
}
fout.close();
fin.close();
cout << "Record with roll number " << rno << " edited successfully" << endl;
remove("recs.dat");
rename("temp.dat", "recs.dat");
}
}
```